

OmniSport

Wielosekcyjny klub sportowy

Sprawozdanie nr.5

Autorzy: Nikodem Kaczmarczyk, Jakub Łukasziak

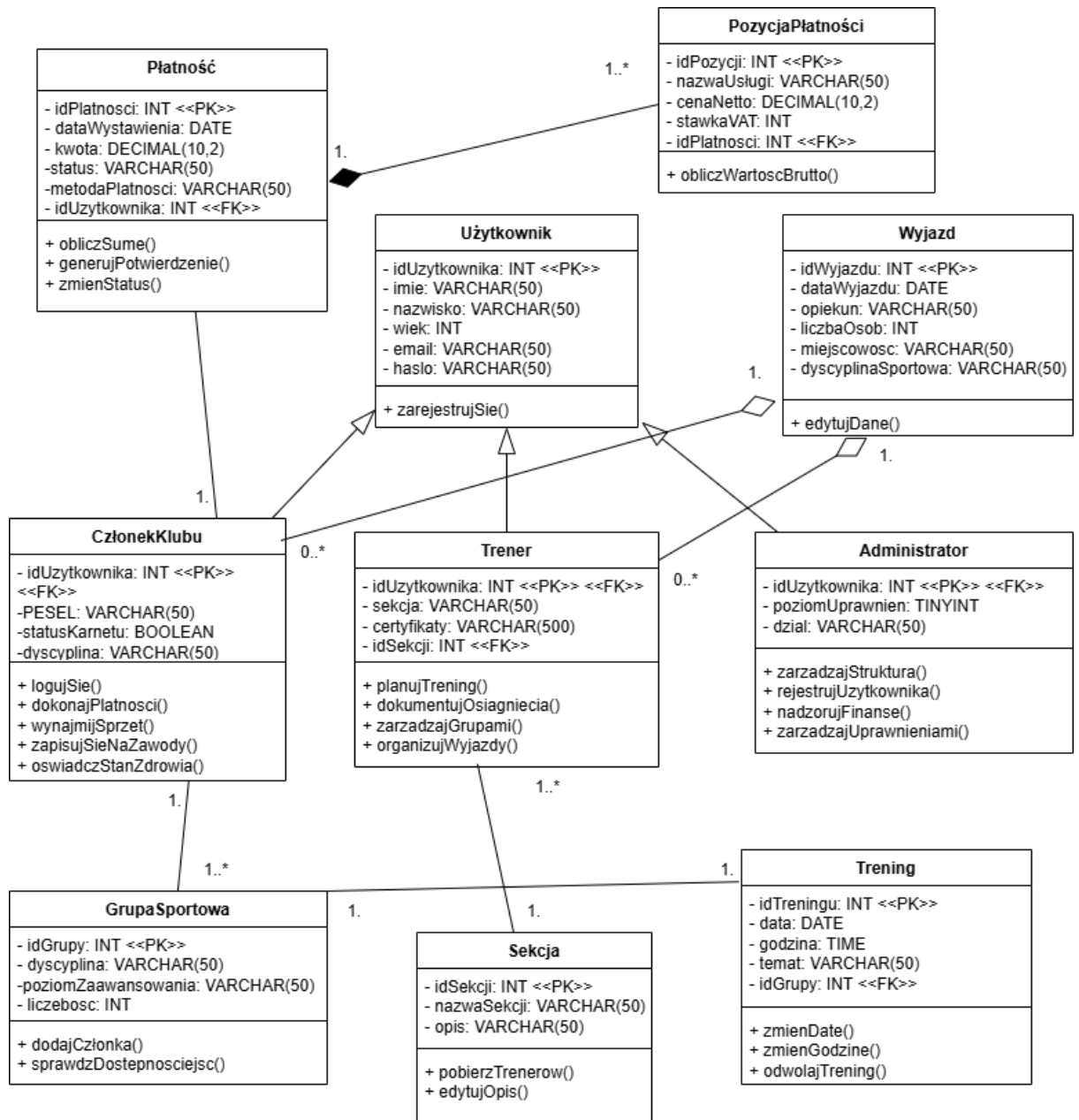
1. Wstęp

Celem niniejszego etapu prac jest przekształcenie konceptualnego modelu danych w projekt implementacyjny. Proces ten obejmuje doprecyzowanie specyfikacji klas o typy danych i widoczność składowych oraz opracowanie schematu relacyjnej bazy danych (RBD) poprzez mapowanie klas UML na tabele.

2. Implementacyjny diagram klas

W celu poprawnego przejścia z modelu obiektowego na relacyjny, stosujemy następujące reguły mapowania:

- **Mapowanie asocjacji 1:N (jeden do wielu):** Klucz główny (Primary Key) tabeli po stronie „1” staje się kluczem obcym (Foreign Key) w tabeli po stronie „N”. Pozwala to na zachowanie więzów integralności między np. Sekcją a przypisanymi do niej Trenerami.
- **Mapowanie kompozycji:** W przypadku relacji Płatność – Pozycja Płatności, klucz główny klasy nadrzędnej wchodzi w całości w skład klucza głównego klasy podrzędnej jako Primary Foreign Key. Stosujemy przy tym regułę **CASCADE**, co oznacza, że usunięcie płatności automatycznie usuwa wszystkie jej pozycje.
- **Mapowanie agregacji:** Realizowane analogicznie do asocjacji, jednak z regułą integralności **SET NULL** (usunięcie "całości" nie usuwa "części", a jedynie czyści powiązanie).
- **Realizacja dziedziczenia:** Związek generalizacji zostaje zastąpiony asocjacją 1:1. Każda klasa (Użytkownik, Trener, Członek Klubu) posiada własną tabelę, a powiązanie między nimi realizowane jest poprzez wspólne klucze główne. Rozwiązanie to optymalizuje zużycie pamięci i ułatwia zarządzanie danymi specyficznymi dla danej roli.



3. Szkielet tabel

- **Użytkownicy** (idUżytkownika: INT <<PK>>, imie: VARCHAR(50), nazwisko: VARCHAR(50), wiek: INT, email: VARCHAR(50), hasło: VARCHAR(50));
- **Trener** (idUżytkownika: INT <<PK>> <<FK>>, sekcja: VARCHAR(50), certyfikaty: VARCHAR(500), idSekcji: INT <<FK>>) ON DELETE CASCADE;
- **CzłonekKlubu** (idUżytkownika: INT <<PK>> <<FK>>, PESEL: VARCHAR(50), statusKarnetu: BOOLEAN, dyscyplina: VARCHAR(50)) ON DELETE CASCADE;
- **Administrator** (idUżytkownika: INT <<PK>> <<FK>>, poziomUprawnien: TINYINT, dział: VARCHAR(50)) ON DELETE CASCADE;

- **Sekcja** (idSekcji: INT <<PK>>, nazwaSekcji: VARCHAR(50), opis: VARCHAR(50)) ON DELETE NO ACTION;
- **Platnosc** (idPlatnosci: INT <<PK>>, idCzlonka: INT <<FK>>, idMetodyPlatnosci: INT <<FK>>, dataWystawienia: DATE, kwota: DECIMAL(10,2), status: VARCHAR(50)) ON DELETE NO ACTION;
- **PozycjaPlatnosci** (idPozycji: INT <<PK>>, idPlatnosci: INT <<FK>>, cenaNetto: DECIMAL(10,2), stawkaVAT: INT, nazwaUslugi: VARCHAR(100)) ON DELETE CASCADE;
- **S_metodyPlatnosci** (idMetody: INT <<PK>>, nazwa: VARCHAR(30));
- **GrupaSportowa** (idGrupy: INT <<PK>>, idTrenera: INT <<FK>>, dyscyplina: VARCHAR(100), poziomZaawansowania: VARCHAR(50), maxLiczbaOsob: INT);
- **Trening** (idTreningu: INT <<PK>>, idGrupy: INT <<FK>>, idTrenera: INT <<FK>>, data: DATE, godzina: TIME, temat: VARCHAR(255));
- **Wyjazd** (idWyjazdu: INT <<PK>>, dataWyjazdu: DATE, opiekun: VARCHAR(100), liczbaOsob: INT, miejscowosc: VARCHAR(100), dyscyplinaSportowa: VARCHAR(100));
- **UczestnicyWyjazdu** (idUczestnictwa: INT <<PK>>, idWyjazdu: INT <<FK>>, idUzytkownika: INT <<FK>>);

4. Mapowanie związków i integralność danych

- **Związki identyfikujące:** Klucz główny tabeli nadrzędnej staje się częścią klucza w tabeli podrzędnej, co wymusza fizyczną zależność istnienia pozycji od dokumentu głównego.
- **Więzy integralności:** Dla wszystkich kluczy obcych zdefiniowano reguły usuwania. W przypadku kompozycji zastosowano CASCADE, natomiast dla asocjacji zwykłych SET NULL lub NO ACTION, aby uniknąć przypadkowej utraty danych historycznych.

5. Normalizacja bazy danych (3PN)

Normalizacja danych to proces tworzenia optymalnej struktury tabel, opartej na idei funkcjonalnej zależności atrybutów od klucza głównego. Celem tego procesu w projekcie OmniSport było wyeliminowanie redundancji (powtarzalności danych) oraz zapobieganie anomalom podczas operacji wstawiania, modyfikacji i usuwania rekordów.

Etapy normalizacji na przykładzie tabeli **Platnosc** i **PozycjaPlatnosci**:

- I Postać Normalna (1PN):

Wszystkie wartości w kolumnach tabel są atomowe i niepodzielne (np. w kolumnie metodaPlatnosci nie przechowujemy kilku wartości naraz). Każda tabela posiada jednoznacznie określony klucz główny (idPlatnosci, idPozycji).

- II Postać Normalna (2PN):

Tabela spełnia warunki 1PN. Wszystkie atrybuty nie kluczowe niosą informację o całym kluczu głównym, a nie o jego części. Jest to widoczne w tabeli PozycjaPlatnosc, gdzie cena i nazwa usługi odnoszą się bezpośrednio do konkretnego identyfikatora pozycji, a nie do całości dokumentu płatności.

- III Postać Normalna (3PN):

Tabela spełnia warunki 2PN. Żaden atrybut niekluczowy nie jest zależny od innego atrybutu nie kluczowego. W naszym projekcie uniknęliśmy błędu polegającego na trzymaniu opisu metody płatności bezpośrednio w tabeli Platnosc. Zamiast tego wprowadziliśmy tabelę słownikową S_MetodyPlatnosc, dzięki czemu zmiana nazwy metody w jednym miejscu nie powoduje niespójności w całej bazie (brak anomalii modyfikacji).

6. Wybór modelu mapowania hierarchii

Zdecydowano się na zastosowanie **Wariantu III** (tabela dla każdej klasy) w celu zachowania najwyższego poziomu czystości strukturalnej i zgodności z 3PN. Rozwiązanie to pozwala uniknąć nadmiarowości danych oraz występowania wartości pustych (NULL) w polach specyficznych dla poszczególnych ról użytkowników (Trener, Członek Klubu, Administrator).

7. Schemat relacyjnej bazy danych (następna strona)

